09 · Capstone-Projekt & Architekturmuster

Architektur begründen, Protokoll und Prüfung festlegen · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie 1 / 32

Das große Finale!

- Neunter Termin: Architektur begründen und unter Bedingungen prüfen
- Plattform wie VL6–VL8: STM32H755, M4 FreeRTOS, M7 Bare Metal
- Vier Ziele: Muster trennen, Datenvertrag erklären, Messpunkte definieren, Abnahme begründen
- Ein beobachtetes Maximum ist kein WCET-Beweis

Notizen 1 / 32

Die bisherigen Termine haben Bausteine geliefert. Heute werden sie zu einer prüfbaren Architektur. EZS ist das Modul Echtzeitsysteme, DHBW die Duale Hochschule Baden-Württemberg. Der Cortex-M4 führt FreeRTOS über CMSIS-RTOS2. CMSIS ist der Cortex Microcontroller Software Interface Standard, RTOS das Echtzeitbetriebssystem. Der Cortex-M7 läuft Bare Metal, also ohne diesen Kernel. MCU ist die Microcontroller Unit, hier der STM32H755. NUCLEO-H755ZI-Q nennen wir nur, wenn das Projekt dieses Board wirklich nutzt.
WCET, Worst-Case Execution Time, ist eine obere Ausführungszeit unter benannten Annahmen. Die Antwortzeit R_i enthält zusätzlich Warten und Interferenz. D_i ist die relative Deadline. „Es lief im Test“ beschreibt gültige Läufe. „Die Frist ist begründet“ braucht das Modell. Cache abzuschalten ändert einen Einfluss und beweist den Determinismus nicht.
Die vier Ziele sind beobachtbar: ein Muster einer Frage zuordnen, den Latest-Value-Vertrag erklären, Messpunkte benennen und eine Abnahme mit offenem Status begründen.
Die Agenda geht von der Architekturentscheidung bis zum Versuchsergebnis.

Folie 2 / 32

Agenda für heute

1. Semesterbausteine und drei Entwurfsprinzipien
2. Edge, AMP, Fail-Safe und Lockstep als verschiedene Fragen
3. AUTOSAR Classic als Schichtenbild, nicht als unser Code
4. Ein Datenvertrag und die Kriterien C1–C8
5. Messpunkte, auch ohne ein bestimmtes Trace-Gerät

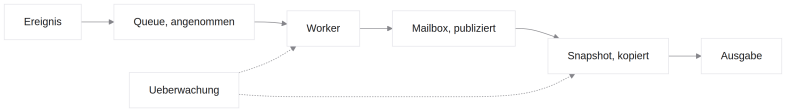
Notizen 2 / 32

Die fünf Blöcke beantworten nacheinander: Was haben wir gebaut, welche Architekturfrage ist welche, wie sieht ein standardisiertes Schichtenbild aus, welchen Vertrag nutzt unser Demonstrator, und woran erkennt man, dass ein Kriterium geprüft wurde. Edge ist der Ort der Verarbeitung. AMP, Asymmetric Multiprocessing, ist die ungleiche Rollenverteilung zweier Kerne. Beides kommt erst im zweiten Block, nicht als Synonym.
PC13 ist ein Tasterpin für einen Funktionstest, kein Hochfrequenztakt. BlueBox ist ein Produktname für einen Recorder, keine Zeitgarantie. Fehlt ein geeigneter gemeinsamer Zeitbezug, bleiben die betroffenen Zeitkriterien ungeprüft. Der Architekturteil kann trotzdem fertig sein. HSEM, Hardware Semaphore, ID 0 bleibt dem Boot, wenn Vorlesung 6 sie so benutzt.
Der Semesterüberblick zeigt den Sensorpfad als Kette von Verträgen.

Folie 3 / 32

Die Reise dieses Semesters

Abbildung A. Jeder Pfeil ist ein Vertrag, kein Automatismus.



Gruppe	Vertrag im Capstone
Erfassen	ISR erkennt, Put mit Timeout 0
Verarbeiten	lokale Queue, Arbeit außerhalb des Schlosses
Austauschen	Latest-Value in SRAM4, HSEM ungleich Boot
Überwachen	Fortschritt, nicht die Tasterrate

Notizen 3 / 32

Die Agenda hat die Reihenfolge genannt. Diese Folie zeigt, wo ein Ereignis angenommen, veröffentlicht, kopiert oder verworfen werden kann. ISR ist die Interrupt Service Routine, EXTI der External Interrupt Controller. Eine Queue ist ein Puffer mit Entnahmereihenfolge. SRAM4 ist die gemeinsame Speicherbank aus Vorlesung 6, nicht ein vierter Kern. Die Überwachung sitzt daneben, weil ein Watchdog keine

Notizen, Fortsetzung 3 / 32

Nutzdatenstation ist.
PC als Program Counter ist die Befehlsadresse im Kern. PC13 ist Port C, Pin 13, der Taster auf vielen Nucleo-Boards. Die beiden Kürzel teilen sich Buchstaben und nicht die Bedeutung. Für den Capstone brauchen wir PC13 nur als möglichen Funktionstest.
Die MPU, Memory Protection Unit, sieht Regionen eines Kerns, nicht jede Arraygrenze und nicht DMA, Direct Memory Access. Der Watchdog sieht nur, ob rechtzeitig und im Fenster aufgefrischt wurde. Ein blinder Feed verdeckt einen Stillstand.
Aus den Bausteinen folgen drei Entwurfsprinzipien.

Folie 4 / 32

Erkenntnis 1: Determinismus schlägt Rohleistung

Eine Super-Loop kann harte Fristen halten, wenn jede relevante Arbeit und jeder Wartepfad ein Budget hat. FreeRTOS beweist das nicht allein. Cache aus verkleinert einen Einfluss und lässt Bus, IRQ und Protokoll stehen.
Jitter benennen wir entweder als Spanne einer festgelegten Antwortzeit oder als Abweichung von einer Sollperiode. Nicht beides mit derselben Zahl.

Notizen 4 / 32

Deterministisch heißt hier: unter genannten Bedingungen bleibt das Verhalten in den begründeten Grenzen. Eine Super-Loop ist eine Endlosschleife ohne Kernel. Sie hält eine Frist, wenn die längste Runde inklusive aller Interrupts in das Budget passt. Ein schneller Kern mit einer langen fremden Busnutzung kann trotzdem zu spät sein. Das ist das Gegenbeispiel zur Rohleistung.
IRQ ist die Interruptanforderung. WCET ist die obere Ausführungszeit eines benannten Stücks, nicht automatisch die Antwortzeit. In Vorlesung 2 war die Rate-Monotonic-Schranke hinreichend unter ihren Annahmen, nicht eine Aussage über jede Anlage. Ein gemessenes Maximum beschreibt die gültigen Läufe. Ohne Analyse der übrigen Pfade ist es keine WCET.
Frage: Warum ist das beobachtete Maximum kein WCET-Beweis? Weil nicht gelaufene Pfade und Interferenzen fehlen können.
Zeitbudgets enthalten auch die Kosten der Synchronisation.

Folie 5 / 32

Synchronisation braucht kurze Abschnitte

- Das Schloss hält die Konsistenz des einen Datensatzes
- Es verspricht nicht, dass jede ältere Fassung den Leser erreicht
- Unter dem Schloss nur schreiben oder kopieren
- Filter, UART und `printf` danach
- Verlorenes Aufwecken aus VL8 bleibt unzulässig

Latest-Value darf Zwischenwerte ersetzen. „Kein Verlust“ wäre ein anderer Vertrag.

Notizen 5 / 32

Der Titel der älteren Folie klang nach einem Verbot. Gemeint ist der Preis: Erwerbswartezeit und Haltezeit. Ein Mutex ordnet Tasks eines Kernels. HSEM, Hardware Semaphore, ordnet den Inter-Core-Besitz, wenn die Kennungen stimmen. Der Producer schreibt einen vollständigen Datensatz. Der Consumer kopiert ihn. Danach rechnen und senden beide außerhalb. UART ist der Universal Asynchronous Receiver/Transmitter. Latest-Value, auch Mailbox genannt, behält den neuesten Stand. Eine ältere Publikation darf überschrieben werden, bevor der M7 sie kopiert. Das ist erlaubt und muss in der Bilanz als Auslassung erscheinen. Eine Queue, die jede Nachricht bis zur Ausgabe behält, ist eine andere Architektur mit Gegendruck. Beides gleichzeitig „verlustfrei“ zu nennen, widerspricht sich.
Genau ein lokaler Nutzer je Core- und ProcessID-Kombination im einfachen Capstone. Weitere Tasks brauchen lokale Serialisierung. Eine HSEM-Notification ist die Meldung einer Freigabe, kein Besitz. Das Warten speichert das Ereignis vor dem Wait, wie in Vorlesung 8.
Schutz- und Diagnosemechanismen sehen nur ihr eigenes Fehlerbild.

Folie 6 / 32

Überwachung wirkt nur mit einem passenden Fehlerbild

Mechanismus	sieht	sieht nicht
Watchdog	ob der Refresh in Zeit und Fenster lag	die fachliche Richtigkeit
MPU	Regionen dieses Kerns	DMA und den anderen Kern
Trace	vollständig angekommene Läufe	die nicht aufgezeichneten

Sensorstille und ein Burst sind erlaubte Lasten. Sie sind kein Hänger und kein Grund, bei jedem Taster zu füttern.

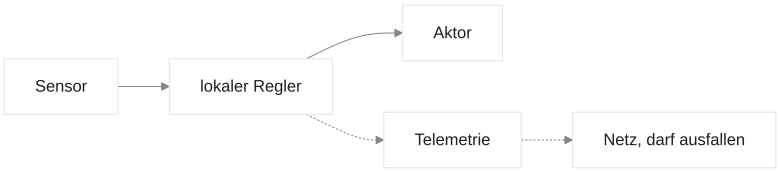
Notizen 6 / 32

Die Schlösser haben die Daten konsistent gehalten. Sie haben nicht geprüft, ob der Worker noch lebt oder das Ergebnis stimmt. Die Software muss die Gesundheitsbedingung liefern. Der Watchdog prüft den zeitlichen Refresh. Die MPU prüft Adressen eines Kerns. Ein Trace ist die Aufzeichnung der Ereignisse, die den Recorder erreicht haben. Ein fehlendes Ereignis ist keine Latenz von null.
WWDG, Window Watchdog, darf nicht an jede Tasterflanke gekoppelt werden. Ein Burst kann zu früh refreshen. Eine legitime Pause kann zu spät wirken, obwohl die Tasks ihre Kontrollrunde beendet haben. Der Supervisor aus Vorlesung 8 prüft frische Fortschrittsnachweise mit eigenen Fristen. Ein weiterlaufender M4-Feed darf einen toten M7 nicht verdecken, wenn dessen Fortschritt zum Vertrag gehört.
Frage: Warum füttert man den WWDG nicht bei jedem Taster? Weil Burst und Stille die Fensterbedingung aus dem falschen Grund treffen.
Vom einzelnen Controller geht es zur Systemarchitektur.

Folie 7 / 32

Vom Mikrocontroller zum CPS

Ein CPS, Cyber-Physical System, koppelt Rechnung und physische Wirkung. Es braucht kein öffentliches Netz.
Abbildung B. Der lokale Kreis hat eigene Fristen. Die Telemetrie ist ein zweiter Pfad.
Edge kann der MCU, ein Gateway oder ein Industrie-PC sein. Edge ist nicht AMP. Eine WAN-Latenz von 50 ms ist ein Anlass, den Pfad zu wählen, keine allgemeine Stabilitätsgrenze für jeden PID-Regler. PID ist der Regler mit proportionalem, integrierendem und differenzierendem Anteil. WAN ist ein Weitverkehrsnetz.
Fällt das Netz aus, bleibt der lokale Kreis im definierten Betrieb. Er stürzt nicht, weil die Telemetrie fehlt.



Notizen 7 / 32

Die Überwachung hat zum Controller gehört. Ein CPS fragt zusätzlich, welcher Pfad die Stellgröße erzeugen darf. Der lokale Sensor-Regler-Aktor-Weg bleibt auf dem Gerät. Die Telemetrie darf Daten nach außen tragen und bei Netzausfall ausfallen, ohne den lokalen Kreis mitzunehmen. 50 ms sind hier eine Lehrannahme für eine langsame Strecke, nicht die Grenze, ab der jeder PID instabil wird. Das hängt von Strecke und Entwurf ab.
Ein fiktives Beispiel, kein Industrienachweis: Der Stromregler taktet lokal im Millisekundenbereich. Dieselbe Stellgröße über eine variable WAN-Strecke wäre der falsche Pfad. Bei Netzausfall setzt die Telemetrie ein Kennzeichen „nicht verbunden“. Die lokale Regelung behält ihren letzten gültigen Betrieb oder einen benannten Ersatz, den die Anwendung festlegt. Wir erfinden dafür keinen Motor.
Die nächste Rechnung zeigt, warum man nicht jede Rohprobe ins Netz schickt.

Folie 8 / 32

Das Edge-Computing Pattern

Lehrannahme, keine Messung: 40 000 Samples je Sekunde, 1 000 Sensoren, 2 Byte je Sample.

$$40000 \times 1000 \times 2 = 8 \cdot 10^7 \text{ Byte/s} = 80 \text{ MB/s}$$

$$80 \times 8 = 640 \text{ Mbit/s}$$

MB ist hier dezimal, 10^6 Byte. Das ist Nutzdatenrate ohne Protokoll, Adressierung und Pausen. Rohdaten, ein Ausschnitt, Merkmale oder Telemetrie sind Entwurfsoptionen. „Nur Metadaten“ ist keine Pflicht. Der zeitkritische Pfad bleibt lokal, auch wenn die Nutzlast sinkt.

Notizen 8 / 32

Die vorige Folie hat den lokalen Kreis vom Netz getrennt. Die Rechnung begründet, warum die Trennung oft nötig ist. 40 000 mal 1 000 mal 2 ist 80 Millionen Byte je Sekunde. Mal 8 sind 640 Millionen Bit je Sekunde, also 640 Mbit/s Nutzdaten. Ein MB mit 10^6 Byte ist nicht das binäre MiB. Die Annahmen stehen nicht auf dem Board. Ein anderes Kompressionsverhältnis oder ein Cloudtarif werden nicht erfunden. Vorverarbeitung am Edge kann Merkmale statt Rohproben senden. Sie darf den Regelkreis nicht hinter das Netz legen, wenn dessen Latenz die lokale Frist verletzt. Abbildung B bleibt gültig: Netzausfall trifft den Telemetriepfeil. Der lokale Pfad läuft weiter, sofern sein eigener Vertrag das vorsieht.
Wo gerechnet wird, ist eine andere Frage als die Rollen der beiden Kerne.

Folie 9 / 32

Asymmetrische Lastverteilung in der Praxis

Abbildung C. Die gestrichelte Linie ist die gewünschte Aufgabengrenze, nicht eine gemessene zeitliche Isolation.



Auf dem M7 läuft kein Linux. Ein Cortex-A-Gateway ist ein anderes Beispiel. Eine M4-Latenz von 10 µs steht hier nicht, solange sie niemand gemessen hat.

Notizen 9 / 32

AMP, Asymmetric Multiprocessing, heißt: die Kerne haben verschiedene Programme und Rollen. Der M4 erfasst mit FreeRTOS. Der M7 filtert Bare Metal und sendet. Edge sagte nur, dass die Verarbeitung nah an der Anlage liegt. Beides kann zusammenfallen und bleibt trotzdem nicht dasselbe. RCC, Reset and Clock Control, gehört zu den geteilten Ressourcen, zusammen mit Bus, Speicher und Teilen der Peripherie. Die gestrichelte Grenze im Text ist diese Rollengrenze. Sie ist keine Mauer gegen Interferenz. Last auf

Notizen, Fortsetzung 9 / 32

dem M7 kann den M4 über den gemeinsamen Bus verzögern, wie in Vorlesung 6. „Gemeinsam ist nur der vereinbarte Block“ gilt später für die Anwendungsdaten, nicht für die Hardware. Frage: Warum sind die Kerne nicht vollständig zeitlich isoliert? Weil sie Bus, Takt und Peripherie teilen.
Eine solche Aufteilung ist noch kein Sicherheitskonzept.

Folie 10 / 32

Safety-Muster: Fail-Safe vs. Fail-Operational

Dieselbe fiktive Anwendung, derselbe benannte Fehler. Links ein definierter Zustand. Rechts eine benannte Restfunktion, befristet und degradiert.
Fail-Safe braucht den sicheren Zustand und die zulässige Reaktionszeit, einschließlich Reset und Boot. „Motor aus“ kann eine hängende Last gefährden. Für den Sensordemonstrator heißt die Reaktion: Ausgabe als ungültig kennzeichnen, keinen Stellantrieb erfinden.
Abbildung D ist ein Schema, kein Nachweis.



Notizen 10 / 32

Safety ist hier der Schutz vor unvertretbaren Folgen, keine Zertifizierung. Fail-Safe führt in einen benannten Zustand. Fail-Operational erhält eine benannte Funktion nach einem benannten Fehler. Ohne diese drei Angaben ist das Wort leer. Ein Watchdog-Reset ist nur dann der sichere Zustand, wenn die Pegel während Reset und die Freigabe nach dem Boot zur Anwendung passen. Pins können hochohmig werden. Das haben wir in Vorlesung 8 festgehalten.
Gegenbeispiel: Eine Last, die beim Abschalten unkontrolliert fällt, ist durch „Ausgang aus“ nicht automatisch sicher. Der Sensordemonstrator hat keinen solchen Aktor. Seine definierte Reaktion ist ein Ungültig-Kennzeichen und keine erfundene Bremse.
Weiterbetrieb braucht oft Redundanz. Deren Aussage muss zuerst klar sein.

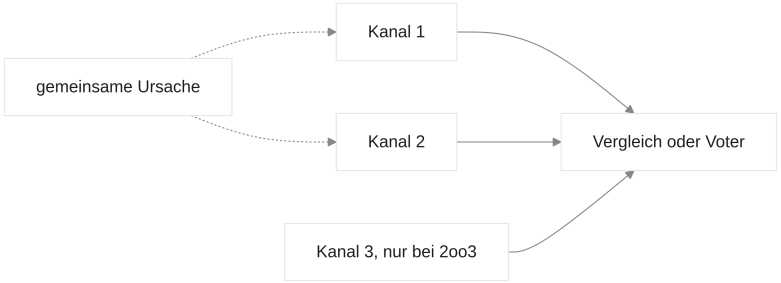
Folie 11 / 32

Redundanz-Architekturen

Zuerst die Entscheidungsgröße: binäre Schutzanforderung oder numerischer Wert.

Muster	liefert	liefert nicht
zwei Kanäle	ein Abweichungszeichen	den richtigen Wert
Active/Standby	einen Umschaltpfad	dass der Standby aktuell ist
2oo3	Maskierung eines Abweichlers	dass die Mehrheit wahr ist

2oo3 heißt two out of three, zwei von drei. Gemeinsame Versorgung, derselbe Anforderungsfehler und ein fehlerhafter Voter treffen alle. Zwei Chiphersteller sind keine Diversität der Software.



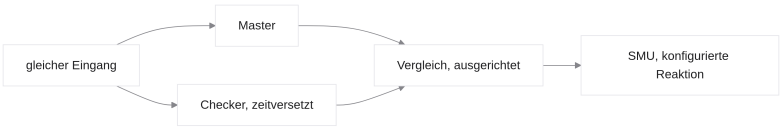
Notizen 11 / 32

Abbildung E trennt Vergleich, Umschaltung und Abstimmung. Zwei Kanäle erkennen, dass sie auseinanderliegen. Ohne weitere Diagnose weiß man nicht, welcher stimmt. Active ist der gerade wirkende Kanal, Standby der bereite. Der Umschaltpfad braucht Fehlererkennung und einen gültigen, hinreichend aktuellen Standbyzustand. Sonst schaltet man auf einen alten Wert. 2oo3 kann unter der Annahme unabhängiger Einzelfehler einen Abweichler überstimmen. Bei analogen Werten braucht der Voter eine Toleranz oder eine Auswahlregel, zum Beispiel einen Median. Die Mehrheit kann gemeinsam falsch liegen. Common Cause ist diese gemeinsame Ursache.
Diversität der Hersteller betrifft das Silizium, nicht automatisch den Algorithmus. Eine unbelegte Flugzeuggeschichte und eine Sicherheitswahrscheinlichkeit fügen wir nicht hinzu. Das Bild bleibt ein Lehrschema.
Lockstep ist eine integrierte Diagnose, kein zweites unabhängiges System.

Folie 12 / 32

Lockstep-Prozessoren (Hardware-Safety)

Abbildung F. AURIX TC3xx, Delayed Lockstep: gleiche Funktion, zeitversetzter Checker, Vergleich nach Ausrichtung, Meldung an die SMU. SMU ist die Safety Management Unit. Die Reaktion ist konfiguriert. Eine Abweichung ist kein automatisch richtiges Ersatzergebnis.
SafeTI ist eine Technologiebezeichnung von Texas Instruments, kein einzelner Prozessorname.
Das H755-Lehrprojekt ist kein Lockstep: M4 und M7 haben verschiedene Rollen.



Notizen 12 / 32

Der Checker rechnet dieselbe Funktion später. Vor dem Vergleich werden die Ergebnisse zeitlich ausgerichtet. Stimmen sie nicht überein, meldet die Logik an die SMU. Welche Reaktion folgt, steht in der Konfiguration, nicht als „immer Halt“ in jedem Chip. Dieselbe systematische Fehlerursache kann beide Läufe gleich falsch machen. Der Vergleich sieht das dann nicht. Lockstep allein ist deshalb nicht Fail-Operational.
Die genaue Verzögerung nennen wir nicht, solange sie nicht aus dem gelesenen AURIX-Handbuch der Familie TC3xx stammt. Infineon beschreibt Delayed Lockstep und die SMU in der Sicherheitsdokumentation dieser Familie. Wir übertragen keine Register auf den STM32.
Frage: Warum ist das H755-AMP kein Lockstep? Weil die Kerne verschiedene Aufgaben rechnen und niemand dieselbe Rechnung zeitversetzt vergleicht.
Die Muster lassen sich vergleichen, ohne sie gleichzusetzen.

Folie 13 / 32

Zusammenfassung Architekturmuster

Frage	Mechanismus	Grenze
Wo rechnen?	Edge	Netzausfall ist ein eigener Pfad
Wer rechnet was?	AMP	gemeinsame Hardware bleibt
Was bleibt nach dem Fehler?	Fail-Safe oder Restfunktion	nur mit benanntem System
Ist dieselbe Rechnung gleich?	Lockstep	kein richtiges Ersatzergebnis

Frage: Warum sind Edge und AMP keine Synonyme? Edge ist der Ort, AMP die Rollenverteilung.
Frage: Was liefert ein Zweikanalvergleich? Ein Abweichungszeichen, nicht den richtigen Wert.

Notizen 13 / 32

Die Tabelle hält die vier Fragen auseinander. Sie können in einer Anlage zusammenkommen und bleiben verschiedene Nachweise. Ein Gateway am Netzrand kann AMP sein und trotzdem keine Lockstep-Redundanz haben. Der H755 bleibt zwei Firmwares mit geteiltem Bus. Der vereinbarte Datenblock ist der Anwendungskanal. Er ist nicht die einzige gemeinsame Ressource.
Die zwei Antworten stehen fest, damit sie in der Diskussion nicht wieder zusammenfallen. Als Nächstes geht es um Software-Schichten und standardisierte Schnittstellen. Das ist ein Exkurs zum Verstehen, kein Anspruch, dass unser Binary AUTOSAR enthält.
AUTOSAR Classic ist der Gegenstand des nächsten Blocks.

Folie 14 / 32

AUTOSAR Classic als Schnittstellenbild

AUTOSAR, AUTomotive Open System ARchitecture, Classic Platform beschreibt Schichten, Schnittstellen und ein konfiguriertes Steuergeräte-Image. Das Labor ist keine AUTOSAR-Implementierung.
Ein fiktiver Sensorbaustein und ein fiktiver Regler tauschen einen benannten Wert über eine Schnittstelle. Der Algorithmus sieht den Port, nicht das Register des gerade eingebauten MCU. Das gelingt nur so weit, wie Konfiguration und Nachweise mitziehen.

Notizen 14 / 32

Der Architekturblock hat Fragen getrennt. Dieser Exkurs zeigt, wie eine Industrie das Schnittstellenproblem organisiert. AUTOSAR Classic ist die Plattform für klassische Steuergeräte. ECU, Electronic Control Unit, ist ein Steuergerät. Die Motivation ist Wiederverwendung: derselbe fachliche Baustein soll nicht bei jedem Mikrocontroller neu an die Register geschrieben werden. Ein fiktives Beispiel genügt. Historische Preise, Mitgliederzahlen und unbelegte Portierungsdauern bleiben draußen.
Unser Capstone bleibt FreeRTOS und Bare Metal mit einem eigenen, kleinen Vertrag. Wer die Schichten später wiedererkennt, soll sie nicht mit der STM32-HAL verwechseln. HAL, Hardware Abstraction Layer, ist die ST-Treiberschicht.
Ohne gemeinsame Schnittstelle koppelt jede Schicht direkt an die nächste Implementierung.

Folie 15 / 32

Warum gemeinsame Schnittstellen helfen

Ein fiktiver Regler liest das Sensorregister selbst, ein zweiter erwartet einen anderen Typ, ein dritter nimmt an, der Wert sei schon über das Netz gefiltert. Dieselben Funktionsnamen reichen nicht. Es fehlen Semantik, Einheit, Gültigkeit und ein Zeitvertrag.
Das ist die Motivation des Exkurses, keine belegte Wirtschaftsakte und keine Projektgeschichte.

Notizen 15 / 32

Die vorige Folie hat das Ziel genannt. Hier steht das Problem ohne eine historische Krise zu behaupten. Drei fiktive Teams koppeln denselben Sensor unterschiedlich: direkte Register, ein `uint16_t` gegen ein `float`, und die Annahme, die Entstörung sei schon auf einem anderen Gerät passiert. Der C-Name `read_sensor` ist dann gleich und die Bedeutung nicht. Wiederverwendung braucht den Vertrag: Einheit, Gültigkeit, Alter und wer schreiben darf.
AUTOSAR beantwortet das mit spezifizierten Schnittstellen und einer Konfiguration je ECU. Die Ausgabe, an der wir das Schichtenbild festmachen, ist die Layered Software Architecture der Classic Platform. Die Plattformübersicht wurde als R25-11 geführt. Für die Lage des Complex Drivers ist die Ausgabe R23-11 im öffentlichen PDF nachvollziehbar. Wir zitieren das Schichtenprinzip, keine ungelesene Klausel.
Die standardisierte Antwort ist das Schichtenbild, nicht eine Firmenchronik.

Folie 16 / 32

Classic-Schichten und die andere Plattform

Classic spezifiziert Schichten und ein weitgehend statisch konfiguriertes OS-Bild. Adaptive ist eine andere Plattform mit ARA, der AUTOSAR Runtime for Adaptive Applications, und spezifizierten POSIX-Bezügen. POSIX ist das Portable Operating System Interface. Adaptive ist nicht „Classic mit Linux“.
OTA, Over-the-Air-Update, ist eine Update-Art. Sie ist keine Eigenschaft, die nur Adaptive hätte.

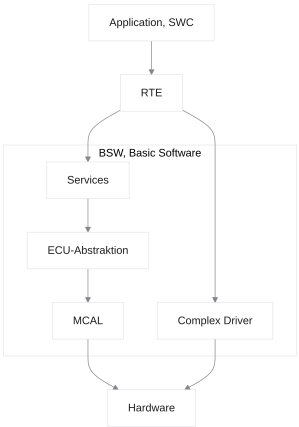
Notizen 16 / 32

Die Problemfolie hat den fehlenden Vertrag gezeigt. Classic antwortet mit Schichten, Schnittstellen und einer Konfiguration, die Tasks, Runnables und Kommunikation vor dem Lauf festlegt. Adaptive richtet sich an größere Plattformen und Dienste. Die Design-Erklärung der Adaptive Platform, Ausgabe R25-11, beschreibt ARA und POSIX-Bezüge. Ein bestimmtes Linux-Gerät oder eine GPU, Graphics Processing Unit, ist keine Definition. Der H755 ist kein Adaptive-Ziel dieses Labors.
OTA kann in beiden Welten vorkommen oder in keiner. Wer es als Alleinstellungsmerkmal von Adaptive nennt, verwechselt eine Funktion mit der Plattformlinie. Für das Schichtenbild bleibt Classic der Gegenstand. Adaptive kommt als Ausblick zurück, nachdem die Schichten stehen.
Das Bild muss die Basic Software sichtbar umranden.

Folie 17 / 32

Das AUTOSAR Schichtenmodell

Abbildung G. Application, RTE, umrandete BSW, Hardware darunter. Der Complex Driver liegt in der BSW, nicht auf Anwendungshöhe.



Rechts daneben, nicht gleichgesetzt: unser Labor aus Algorithmus, schmaler eigener Schnittstelle, HAL oder FreeRTOS und Hardware. STM32-HAL ist kein AUTOSAR-MCAL.

Notizen 17 / 32

BSW ist die Basic Software. MCAL, Microcontroller Abstraction Layer, ist darin die hardwarenahe Schicht, nicht eine vierte Hauptschicht neben der BSW. RTE, Runtime Environment, sitzt zwischen Anwendung und BSW. Ein Complex Driver ist der Sonderpfad in der BSW: eng an Hardware und Anwendung, aber nicht eine Softwarekomponente oberhalb der RTE. Die öffentliche Layered Software Architecture, Classic R23-11, zeichnet diesen Bereich in der BSW. Nicht jeder Pfeil ist ein zwingender serieller Aufruf durch alle Kästen. Lokale Kommunikation kann kürzer sein.
Das Laborbild hat weniger Kästen und einen anderen Namen. Portabilität einer Schnittstelle ersetzt nicht OS-Port, Linker, Konfiguration und Timingnachweis. Ein MCU-Wechsel kann all das mitziehen. „Nur ein paar Dateien tauschen“ ist zu wenig, und „vollständig portabel“ wäre zu viel.
Die nächste Folie bleibt bei den Aufgaben innerhalb der BSW.

Folie 18 / 32

MCAL als Teil der BSW

Services, ECU-Abstraktion und MCAL sind Teile der BSW. Services sind nicht die ganze BSW. Die RTE sitzt darüber.
Ein Sensorzugriff läuft vom hardwarenahen Treiber über die Abstraktion zu einer Schnittstelle, die der Algorithmus nutzen kann. Dieselbe Schnittstelle erspart nicht Integration und nicht den Nachweis der Frist.

Notizen 18 / 32

Der Titel nennt MCAL und BSW zusammen, weil die MCAL in der BSW liegt. ECU ist das Steuergerät, dessen Abstraktionsschicht Zugriffe bündelt, die nicht mehr das einzelne Register zeigen. Ein fiktiver Pfad: Der MCAL-Treiber liest das Peripherieregister. Die ECU-Abstraktion bietet eine kanalbezogene Funktion. Ein Service kann Diagnose oder Speicher dazunehmen. Die RTE stellt der Anwendung den Port bereit. Wer nur die Services zeichnet, hat die BSW zu klein gezeichnet.
Eine standardisierte Schnittstelle fördert Wiederverwendung. Sie beweist nicht, dass der Stack reicht oder die Deadline hält. Beim Wechsel des MCU prüfen wir OS-Port, Konfiguration, Complex Driver, Linker und die Zeitanalyse erneut. Die STM32-HAL kann eine ähnliche Rolle wie ein Treiber spielen. Sie wird dadurch nicht zur AUTOSAR-MCAL unseres nicht vorhandenen Stacks.
Das Betriebssystem ist die nächste Vergleichsebene, nicht eine weitere Hardwarezeile.

Folie 19 / 32

Das AUTOSAR OS und FreeRTOS

Classic konfiguriert OS-Objekte statisch. Das ist keine Aussage über jede Adaptive-Plattform und kein Beweis, dass der Stack nie überläuft.
FreeRTOS kann Objekte statisch anlegen. `xTaskCreateStatic` zählt die Stacktiefe in `StackType_t`-Elementen und braucht die statische Allokation im Kernel. `osThreadNew` ist CMSIS-RTOS2. `osThreadAttr_t.stack_size` ist in Bytes. Der Aufruf allein zeigt nicht, ob Control Block und Stack statisch bereitstehen.
Beispiel, portabhängig: 256 `StackType_t` zu 4 Byte sind 1024 Byte. `stack_size = 256` sind 256 Byte. Dieselbe Zahl ist nicht dieselbe Größe.

Notizen 19 / 32

Statische Konfiguration legt die Objekte vor dem Lauf fest. Statische Speicherbereitstellung heißt: der Speicher für Task und Control Block wird vom Programm gestellt, nicht zur Laufzeit aus einem Heap geholt. Beides kann zusammenfallen und ist nicht dasselbe Wort. TCB, Task Control Block, sind die Verwaltungsdaten. Ihre Größe hängt vom Port ab. Wir übernehmen keine feste Bytezahl aus einem fremden RTX-Port.
Auf dem Cortex-M ist `StackType_t` im üblichen Port 32 Bit. 256 Elemente sind dann 1024 Byte. Die CMSIS-Angabe in Bytes würde für denselben Stack 1024 lauten, nicht 256. Ob der konkrete Port abweicht, steht im Adapter. `osThreadNew` ohne `stack_mem` und `cb_mem` beweist die statische Bereitstellung nicht. Ein reservierter Stack beweist weder, dass er reicht, noch eine Deadline.
Die RTE bildet die fachlichen Einheiten auf dieses OS ab.

Folie 20 / 32

Die RTE und das Runnable-Mapping

Die VFB, Virtual Functional Bus, ist das abstrakte Kommunikationsbild. Die RTE ist die ECU-spezifische Realisierung, oft generiertes C. Sie packt nicht allein CAN-Rahmen. CAN ist das Controller Area Network. Entfernte Sender-Receiver laufen durch die konfigurierte BSW. Lokal kann der Zugriff direkt sein. Client-Server ist das Dienstmodell, nicht dasselbe wie ein Datenport. Eine SWC, Software Component, ist keine Task. Ein Runnable ist die ausführbare Einheit, ausgelöst durch ein Ereignis. Die Zuordnung auf Tasks ist fiktiv im folgenden Bild.

Notizen 20 / 32

Abbildung H hat zwei Teile. Oben sprechen zwei SWCs über die VFB. Darunter einmal dieselbe ECU, einmal zwei ECUs mit Netz. Die Netzlatenz, die Puffer und die Fehler der BSW bleiben in der Analyse. Unten ein fiktives Mapping: drei Runnables auf zwei Tasks. Zwei können in einer Task nacheinander liegen, das dritte in einer anderen. Die SWC bleibt die logische Komponente. Sie wird nicht zur FreeRTOS-Task, nur weil ein Runnable dort läuft.

Notizen, Fortsetzung 20 / 32

Sender-Receiver transportiert Daten. Client-Server ruft einen Dienst und kann auf die Antwort warten. Ein gleicher Portname auf einer anderen Verteilung hat nicht dieselbe Antwortzeit. Datenalter und Queue-Tiefe rechnen wir neu.
Frage: Ist eine SWC eine Task? Nein. Runnables werden auf Tasks abgebildet.
Der Algorithmus sieht danach nur noch seine Ports.

Folie 21 / 32

Application Layer und SWC



Der fiktive Regler liest einen Sensorport, prüft Gültigkeit und Alter und liefert ein Ergebnis. Das Register bleibt unter der RTE. Derselbe Portname auf zwei ECUs garantiert nicht dieselbe Latenz.

Notizen 21 / 32

Ein Port ist der Anschluss einer SWC, nicht der GPIO-Port des STM32. Die Daten- oder Dienstschnittstelle sagt, was hindurchgeht. Das Runnable „prüfen“ sieht den Wert und sein Alter. Es sieht nicht, ob der MCAL-Treiber gerade den Bus belegt. Hardwaretreiber bleiben in der BSW. Wechselt die Verteilung von einer ECU auf zwei, kann zwischen den Runnables ein Netzwerkpfad liegen. Die Analyse von Datenalter und Frist beginnt neu, auch wenn der C-Name gleich bleibt.
Das Bild ist fiktiv und klein genug, um die Zuordnung zu zeigen: zwei Runnables in Task 1, eines in Task 2. Eine dritte Task wäre nur ein weiteres Beispiel, keine Vorschrift. Für unser Labor bleibt die Lehre: die schmale Schnittstelle ersetzt nicht den Zeitvertrag.
Adaptive ist die andere Plattformlinie, kein größerer Classic-Stapel.

Folie 22 / 32

Ausblick: AUTOSAR Adaptive

Ebene	Classic, dieser Exkurs	Adaptive, Ausblick
Ziel	Steuergerät, oft MCU	größere Plattform
Kommunikation	RTE, BSW, auch CAN	ARA-Dienste
OS-Bezug	konfiguriertes Classic-OS	POSIX-Bezug, nicht unser FreeRTOS
Integration	Konfiguration und Nachweis	andere Prozesse und Updates

FreeRTOS ist ein Kernel. AUTOSAR ist ein Satz von Standards. Keine Rangliste.

Notizen 22 / 32

ARA bündelt die Laufzeitdienste der Adaptive Platform. POSIX-Bezüge betreffen Schnittstellen, die ein passendes Betriebssystem bereitstellt. Typische Ziele sind größere Prozessoren, oft Cortex-A. Ein bestimmtes Linux, eine GPU oder „alles wird dynamisch allokiert“ ist nicht die Definition. Der H755-Demonstrator wechselt dadurch nicht den Modus.
Die Tabelle vergleicht Ebenen, nicht den Sieger. Classic-OS, FreeRTOS und Adaptive nebeneinanderzulegen, ohne die Ebene zu nennen, erzeugt Scheinwidersprüche. Unser Kernel kann statische Objekte anlegen. Das macht ihn nicht zum Classic-OS. OTA bleibt eine Funktion, die beide Linien betreffen kann. Die Kursentscheidung begründet, warum wir den kleinen Vertrag selbst schreiben.

Folie 23 / 32

Warum der Capstone klein bleibt

Für dieses Labor ist die Einarbeitung in einen AUTOSAR-Stack größer als der Gewinn der fertigen Schnittstelle. Das ist eine Kursentscheidung, keine Marktstudie.
Übertragbar bleiben: eine schmale Schnittstelle, ein benannter Speichervertrag, kurze kritische Abschnitte und eine definierte Zeit- und Fehlersemantik. Nicht übertragbar ist der Satz, ein zusätzlicher Layer mache den Demonstrator robust.

Notizen 23 / 32

Die Vergleichstabelle hat die Standards neben unseren Kernel gestellt. Die Entscheidung folgt aus dem Lernziel: die Studierenden sollen den Vertrag selbst festlegen und prüfen können. Ein abstrakter Layer, dessen Konfiguration niemand im Kurs liest, würde diese Arbeit verdecken. Kosten in Geld oder Personenwochen nennen wir nicht.
Was bleibt, ist in den nächsten Folien konkret. Die Schnittstelle ist die Mailbox mit Gültigkeit, Kennung und

Notizen, Fortsetzung 23 / 32

Zeitpunkt. Der Speichervertrag ist der aus Vorlesung 6: eine nicht cachebare Region, ein Initialisierer, Barrieren. Die kurzen Abschnitte sind Schreiben und Kopieren unter HSEM. Die Fehlersemantik sind Queue-Drops, Mailbox-Auslassungen und ein Supervisor, der nicht an der Tasterrate hängt.
Damit endet der Exkurs bei den übertragbaren Regeln, nicht bei einem Produkturteil.

Folie 24 / 32

Fazit des Schnittstellenexkurses

Classic erklärt Schichten, VFB und RTE sowie die Abbildung von Runnables auf Tasks. Adaptive ist die andere Linie. Der Capstone bleibt FreeRTOS auf dem M4 und Bare Metal auf dem M7, mit einem benannten Protokoll.
Frage: Wo liegen MCAL und Complex Driver? In der BSW, unter der RTE. Frage: Warum sind 256 Stackelemente nicht 256 Byte? Weil FreeRTOS `StackType_t` zählt und CMSIS Bytes.

Notizen 24 / 32

Die vier Sätze fassen den Exkurs, ohne ihn zu wiederholen. Die RTE realisiert die VFB auf einer ECU. Runnables sind keine Tasks. Classic und Adaptive bleiben getrennte Plattformen. Unser Binary enthält davon nichts. Die gleichen Schnittstellenfragen kehren jetzt als eigener kleiner Vertrag zurück: Was wird veröffentlicht, was darf verloren gehen, woran misst man die Zeit, und wann ist ein Kriterium geprüft.
Die zwei Antworten verhindern die häufigsten Verwechslungen aus diesem Block. Als Nächstes stehen die Stufen des Capstone, nicht noch eine Schicht.
Stufe A, B und C bekommen je eine Bedeutung.

Folie 25 / 32

Das Capstone-Projekt

Drei Stufen, eine Bedeutung je Buchstabe:

Stufe	Inhalt	Zeitkriterium
A	M4: Erfassung, Queue, Zähler, Watchdogrechnung	Vorbereitung
B	beide Kerne, Latest-Value und HSEM	Architekturpflicht
C	geeignete Messung der geforderten Größen	sonst „nicht geprüft“

C ist keine Pflicht, ein bestimmtes BlueBox-Modell zu besitzen. Ohne gemeinsamen Zeitbezug bleibt das Zeitkriterium ungeprüft. Ungeprüft ist nicht bestanden und nicht automatisch verfehlt.

Notizen 25 / 32

Die Stufen lösen den Widerspruch, Multicore und Trace seien gleichzeitig Muss und Erweiterung. A ist der M4-Grunddemonstrator: Ereignisse, Queue, Fehlerzähler und eine dokumentierte Watchdogrechnung. B ist der eigentliche Capstone: M7 und M4 mit dem Mailboxvertrag. C ist die instrumentierte Prüfung. Ein Diagramm oder eine Rechnung macht C nicht zu einem bestandenen Hardwaretest.
Liefergegenstände, auch ohne Board: Konfigurationsblatt, Datenvertrag, Ablauf von Producer und Consumer, die Tabelle C1–C8, eine Testmatrix und ein Ergebnisblatt mit leeren Beobachtungsfeldern. Statusworte sind: geplant oder nicht geprüft, bestanden unter Bedingungen, verletzt, Messung ungültig, nicht anwendbar mit Begründung.
Der Kern von B ist der Datenvertrag. Der Eingang beschreibt, welche Versuche ihn treffen.

Folie 26 / 32

Der Smart-Sensor: drei Versuche

PC13, wenn das Board den Taster dort hat: Funktion und Entprellung. Keine Hochfrequenzabnahme. Der Timer erzeugt die reproduzierbare Folge. Ein externer Generator mit Input Capture misst die physische Flanke. EXTI-Pending kann Flanken zusammenfassen. Zehn physische Flanken sind daher nicht automatisch zehn Put-Versuche.
Lehrfestlegungen, keine Messwerte: Grundlast 100 Hz, also 10 ms Abstand. Burst: zehn Ereignisse bei 0, 1, …, 9 ms. Queue-Sättigung: Tiefe 8, anfangs leer, zehn Put-Versuche ohne Get. Dann genau acht Annahmen und zwei Drops, wenn jeder Put korrekt war.
Entprellung nur im Tasterversuch, in der Task, als eigener Verwurf. Im Timerversuch keine versteckte Entprellung.

Notizen 26 / 32

Drei Versuche dürfen nicht in einen Topf. Die Grundlast prüft den Dauerbetrieb. Der Burst prüft eine kurze dichte Folge. Ob der Worker zwischen den Millisekunden schon entnimmt, hängt von Priorität und Laufzeit ab. Mindestens zwei Drops folgen daraus nicht automatisch. Die Sättigung schaltet das Get während der zehn Puts ab und zählt dann. Eine Vorbelegung oder ein weiterer Fehler kann mehr Abweisungen erzeugen. `osMessageQueuePut` mit Timeout 0 in der ISR unterscheidet volle Queue, ungültiges Objekt und eine unzulässige Priorität. Im regulären Lehrlauf erwarten wir nur die vollen-Queue-Drops, die anderen Fehlerzähler bei null.
Die Bilanz, die hier gilt, ist nicht „angenommen gleich Drops plus Queue plus letzter Snapshot“. Ein Snapshot ist ein Datensatz. Die Zähler stehen auf der Kriterienfolie. Schon hier gilt: Erfasstes I teilt sich in angenommen A, Queue-Drops D_q und sonstige Put-Fehler. A teilt sich in noch liegend Q, in Bearbeitung F, veröffentlicht P und Taskverwurf D_{task}. Beim Taster kann D_{task} die Entprellung sein. Beim Timer bleibt sie null, solange nichts anderes verworfen wird.
Die Architektur zeigt, wo diese Zähler sitzen.

Folie 27 / 32

System-Architektur des Capstone

Abbildung I. Die Queue ist lokal auf dem M4. Filter und UART liegen außerhalb des HSEM.



Lehrziele unter benannter Last, keine Messung: Publikation von ISR-Eintritt bis Freigabe höchstens 1 ms. Alter des wirklich kopierten neuen Snapshots höchstens 50 ms. Die Ausgabe bis UART ist eine eigene Größe.

Notizen 27 / 32

Producer und Consumer sind nicht dieselbe Operation. Der Worker veröffentlicht unter der Nutzdaten-HSEM. Der M7 kopiert unter derselben ID und filtert danach. Die gestrichelte Überwachung ist keine Station der Nutzdaten. C3 betrifft die Publikationsantwort angenommener Nachrichten, die zur Veröffentlichung vorgesehen sind. C4 betrifft das Alter der kopierten Publikation und braucht einen gemeinsamen Zeitbezug. Ein GPIO-Puls um Take und Release vermischt Erwerbswarten und Haltezeit. Die Haltezeit beginnt nach bestätigtem Besitz.
Die 1 ms und 50 ms sind Lehrfestlegungen für die Normallast, nicht für den Fall „M7 hält das Schloss und hängt“. Dort gilt ein Timeout- und Recoverypfad, keine bestandene 1-ms-Prüfung. UART-Blockierung auf dem M7 kann die Snapshotrate senken und damit erlaubte Latest-Value-Auslassungen erhöhen. Die müssen bilanziert werden, nicht als verbotener Verlust und nicht als Null-Latenz.
Die Kriterien machen daraus eine Tabelle ohne die falsche Snapshot-Summe.

Folie 28 / 32

Capstone-Pflichtenheft C1–C8

ID	Lehrgrenze	gilt für
C1	100 Hz und getrennt der 1-ms-Burst	Beschreibung der Last, nicht eine Flanke gleich ein Put
C2	Tiefe 8; Sättigung: 8 Annahmen, 2 Drops	leere Queue, kein Get, korrekte Puts
C3	$R_{\text{pub}} = t_4 - t_1 \leq 1\,\text{ms}$	angenommene, zur Publikation vorgesehene Events, Normallast
C4	$A_{\text{snap}} = t_5 - t_1 \leq 50\,\text{ms}$	kopierte neue Publikationen, plus Stale-Regel und Abdeckung
C5	Boot-ID reserviert, Nutz-ID frei und geprüft	nicht „Boot-ID benutzen“
C6	Haltezeit $\leq 50\,\mu\text{s}$ je Seite	nach Erwerb, ohne Wartezeit
C7	Fenster aus PCLK, nicht aus dem Taster	früh, gültig, spät
C8	zwei Bilanzen, kein Snapshot als Summe	kohärenter Zeitpunkt

Abbildung K. Lehrsequenz, keine Boardmessung: zehn Put-Versuche, acht Annahmen, zwei Queue-Drops. M7 kann weniger als acht Snapshots sehen. Das ist O, nicht ein Fehler von C2.

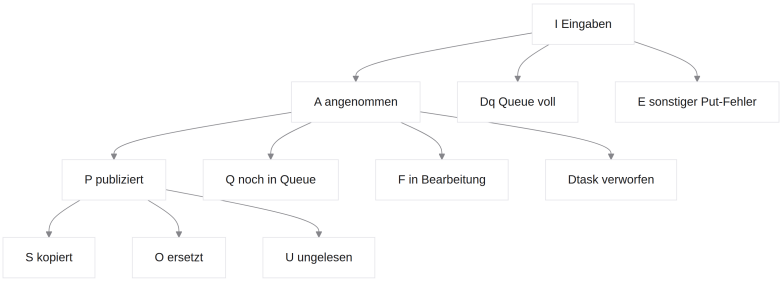


Abbildung J. Unter dem Lock nur der vollständige Satz. Filter danach. Eine neue Publikation darf eine unbestätigte alte ersetzen.

Notizen 28 / 32

I sind bilanzierte Eingabeereignisse der ISR, nicht automatisch jede physische Flanke. A wurde in die Queue angenommen, D_q wegen voller Queue abgewiesen, E_{put} sind andere Put-Fehler und im regulären Lauf 0. Dann gilt $I = A + D_q + E_{\text{put}}$. Q liegt noch in der Queue, F ist entnommen und noch nicht abgeschlossen, P ist erfolgreich veröffentlicht, D_{task} danach verworfen. Dann gilt $A = Q+F+P+D_{\text{task}}$. Nach einem sauber beendeten Versuch sind Q und F null. Im Beispiel ohne Taskverwurf: $I = 10$, $A = 8$, $D_q = 2$, $P = 8$.

Die Mailbox getrennt: $P = S + O + U$. S hat der M7 verschieden kopiert, O wurde ersetzt, bevor es ein erstes Mal kopiert wurde, U ist die noch ungelesene Publikation, 0 oder 1. Ein Ack unter demselben Lock kann das tragen: der M7 bestätigt die kopierte Publikationsnummer, der M4 sieht vor dem Überschreiben, ob die alte schon bestätigt war. Das ist ein Lehrverfahren, kein vorhandener Code. Eine Lücke in der

Notizen, Fortsetzung 28 / 32

Ereignisnummer allein trennt Queue-Drop und Mailbox-Auslassung nicht, wenn die Nummer vor dem Put vergeben wird. Eine eigene Publikationsnummer steigt nur bei erfolgreicher Veröffentlichung.
C6: T_{wait} ist Bitte bis Besitz, T_{hold} ist Besitz bis Freigabe. Der GPIO von vor Take bis nach Release kann beides enthalten. Die 50 µs sind die Lehrgrenze der Haltezeit für Producer und Consumer, mit dokumentierter Marke und Unsicherheit. Eine 1-ms-Tickuhr löst das nicht auf. C5 reserviert die Boot-ID. C7 übernimmt die Lehrkonfiguration aus Vorlesung 8, wenn kein Projekt-PCLK da ist: WWDG2, Teiler 8, Reload 127, Window 80, PCLK-Annahme 100 MHz. Der Feed hängt am frischen Nachweis, nicht am Taster.
Die Abnahme trennt diese Grenzen von leeren Beobachtungsfeldern.

Folie 29 / 32

Abnahmekriterien

Definition of Done heißt hier: je Kriterium ein Test, eine Beobachtung und ein Status. Nicht „kein Verlust“, sondern keine ungeklärte Lücke. Erlaubte Queue-Drops und erlaubte Mailbox-Auslassungen stehen in der Bilanz.

Kriterium	Stufe	Normallast-Erwartung	Status ohne Lauf
C1, C2, C7-Rechnung	A	wie Tabelle	nicht geprüft
C3–C6, C8	B	Vertrag, Messung in C	nicht geprüft
Zeitgrößen	C	gültige Paare, Abdeckung	nicht geprüft

„M7 hält HSEM und hängt“ ist Fehlerinjektion. Dort gilt Timeout und Recovery, nicht die 1 ms. „Queue voll“ erwartet die Drops, nicht gleichzeitig null Verlust.

Notizen 29 / 32

Jedes Kriterium bekommt Test-ID, Last, Konfiguration, analytische Erwartung, Artefakt, beobachtete Größe, Zahl gültiger Stichproben und einen Status. Die Beobachtung bleibt leer, bis ein Lauf sie füllt. Ein Architekturdiagramm füllt sie nicht. Für C3 und C4 zählen nur korrelierte Kennungen. Fehlende t_5 sind keine Latenz null und kein Grund, die Publikation aus O zu streichen, ohne sie zu berichten. Min, Median, ein benanntes Perzentil und das beobachtete Maximum brauchen die Anzahl und die Methode. Ein Prozent aus wenigen Samples ist kein harter Zeitnachweis.
Die Testmatrix trennt Normallast und Fehlerinjektion: Ruhe, Einzelevent, 100 Hz, 1-ms-Burst, kontrollierte Sättigung, M7-Last, UART-Last, kurze HSEM-Kollision, M7 hängt, M4 hängt, Refresh früh, gültig und spät. Optional ein MPU-Versuch mit eigener Recovery, weil die MPU keine Task beendet. M7-Last darf den M4 über den Bus beeinflussen. Das ist ein mögliches Ergebnis, kein Fehler der Anleitung. Durch alle Lasten beobachtet zu haben, beweist keine universelle Deadlockfreiheit.
Ohne geeignete Aufnahme bleibt das Zeitkriterium nicht geprüft. Der Bericht nennt das so.
Die Messpunkte machen die Paare eindeutig.

Folie 30 / 32

Labor-Checkliste: Messpunkte

Abbildung L. t_3 ist der Verarbeitungsbeginn dieser Nachricht, nicht der einmalige Taskstart. t_5 gibt es nur für kopierte Publikationen.

Marke	Bedeutung
t_0	physische Flanke, nur mit Capture-Bezug
t_1	Beginn der M4-ISR für dieses Ereignis
t_2	nach erfolgreichem Put dieser Nachricht
t_3	erfolgreiches Get, Beginn dieser Bearbeitung
t_4	dokumentierte Freigabe der Publikation
t_5	Abschluss des M7-Snapshots dieser Publikation
t_6	Filter zu derselben Kennung fertig
t_7	definierter UART-Abschluss, nicht der Snapshot

$R_{\text{pub}} = t_4 - t_1$. $A_{\text{snap}} = t_5 - t_1$ nur für kopierte neue Stände. Zwei lokale DWT-Zähler werden nicht subtrahiert. Eine Haltezeit von $50\,\mu\text{s}$ braucht eine feinere Uhr als den 1-ms-Tick.



Notizen 30 / 32

DWT, Data Watchpoint and Trace, und CYCCNT, der lokale Zykluszähler, messen Intervalle eines Kerns bei bekanntem Takt. Sie sind ohne belegte Korrelation keine gemeinsame Uhr von M4 und M7. Für $t_5 - t_1$ braucht es eine der drei offenen Methoden: gemeinsame externe Aufnahme, ein beiden Kernen zugänglicher Timer mit einem Takt, oder eine korrelierte Traceaufnahme. Welche Instanz das Projekt hat, steht nicht im Repo. Bis dahin bleibt C4 nicht geprüft. Ein vom M4 gespeicherter Tick minus ein M7-Tick ist keine dieser Methoden.

`HAL_HSEM_Release` liefert in der üblichen HAL keinen Erfolgscode. Die Marke für t_4 liegt an einer dokumentierten Stelle relativ zum Release. Eine Marke danach kann sich schon mit dem anderen Kern überlappen. Sie erzwingt keine falsche Gesamtordnung. t_7 ist entweder die Softwareübergabe, das DMA-Ende oder das Ende des letzten Bits. Der DMA-Transfer kann fertig sein, während der UART noch sendet. Für 8N1, acht Datenbits, keine Parität, ein Stopbit plus Startbit, dauert die Leitung analytisch etwa $10L$ Bitzeiten. Baudrate und Paketlänge sind Projektparameter, keine Folienkonstanten. Die 50 ms des Snapshots ersetzen ein Ausgabeziel nicht.

Ein Queue-Drop endet vor t_3. Eine ersetzte Publikation hat kein t_5. Dieselbe alte Publikation nach Sensorstille wird als stale gekennzeichnet, nicht als frische Messung. Stille ist kein CPU-Hänger. Der Watchdog bleibt an den Fortschrittsfristen. Debug-Freeze aus, wenn der Reset geprüft wird. EWI, Early Wakeup Interrupt, setzt einen GPIO-Puls, kein `printf`. PB14 und LD3 nur, wenn UM2408 und die Bestückung den Pin so zeigen.

Die Schlussfolie bindet die Ziele an die Artefakte, die abzugeben sind.

Folie 31 / 32

Die Essenz des Capstone

- Muster bleiben verschiedene Fragen: Ort, Rolle, Fehlerreaktion, gleiche Rechnung
- Gemeinsame Anwendungsdaten sind der Mailbox-Block. Bus, Takt und Reset bleiben geteilt
- Latest-Value erlaubt benannte Auslassungen. Ungeklärte Lücken sind keine davon
- Eine Frist gilt nur mit Messpunkt, Abdeckung und Status. Sonst: nicht geprüft

Frage: Warum ist eine Notification kein Besitz? Weil danach neu genommen werden muss.
Frage: Warum ist ein Snapshot keine Verlustbilanz? Weil er ein Datensatz ist, keine Summe.
Frage: Warum ist das GPIO um Take nicht immer die Haltezeit? Weil das Warten vor dem Besitz darin stecken kann.

Notizen 31 / 32

Die vier Lernziele haben Antworten. Muster lassen sich trennen. Der Vertrag ist Latest-Value mit Gültigkeit, Ereignis- und bei Bedarf Publikationsnummer, Zeitpunkt, kurzem Schreiben und kurzem Kopieren. Die Messpunkte t_0 bis t_7 sind je Ereignis oder je Publikation definiert. Die Abnahme nennt Status und leere Beobachtung, solange kein Lauf da ist. „Gemeinsam ist nur der vereinbarte Block“ gilt für diese Anwendungsdaten. Die Hardwareinterferenz aus der AMP-Folie bleibt.
Abzugeben sind das Konfigurationsblatt, der Datenvertrag, die Kriterien C1–C8, die Testmatrix und das Ergebnisblatt. Offene Felder: Boardrevision, HAL- und Portstand, echte Taktwerte, NVIC-Prioritäten, freie HSEM-ID, UART-Parameter und die gemeinsame Zeitbasis. Quellen der Erklärung, nicht als Firmware nachgewiesen: AUTOSAR Classic Layered Software Architecture R23-11 für das Schichtenbild, Adaptive Platform Design R25-11, CMSIS-RTOS2 für Queue und `stack_size`, FreeRTOS `xTaskCreateStatic`, AN5557 Rev. 3 zur Dual-Core-Architektur, RM0399 und das H755-Datenblatt in der gelesenen Revision, AURIX TC3xx für Lockstep und SMU. Im Repo liegt kein Firmwarebuild und keine Messaufnahme.
Der nächste Arbeitsschritt ist das Ausfüllen der Blätter für den eigenen Versuch, nicht ein behaupteter Hardwaretest.

Folie 32 / 32

Datenvertrag und Glossar, zu Folie 31

Lehrstruktur, ungebaut, ein Initialisierer, Region wie VL6 nicht cachebar. Das Ping-Pong-`ipc` ist ein anderes Protokoll und wird nicht gleichzeitig auf dieselben Felder gelegt.

```
typedef struct {
    uint32_t valid;
    uint32_t event_seq;
    uint32_t pub_seq;
    uint32_t event_tick;
    uint32_t payload;
    uint32_t ack_pub; /* M7 bestaetigt, Lehrverfahren */
} cap_box_t;
```

Kurz	hier
I, A, D_q, F, P	Queuebilanz
S, O, U	Mailboxbilanz
$t_4 - t_1$	Publikation, Lehrziel 1 ms
$t_5 - t_1$	Alter kopierter Stände, Lehrziel 50 ms
Haltezeit	nach Erwerb, Lehrziel 50 µs

Notizen 32 / 32

Dieser Anhang gehört zur Schlussfolie. `valid` sagt, ob der Stand lesbar ist. `event_seq` entsteht vor dem Put und kann Lücken aus Drops und aus späterem Überschreiben enthalten. `pub_seq` steigt nur bei gelungener Veröffentlichung. `ack_pub` lässt den Producer erkennen, ob die alte Publikation schon kopiert wurde. Wiederholtes Kopieren derselben Nummer erhöht S nicht. Nach Reset braucht die Versuchsgeneration einen definierten Anfang, sonst sieht eine Null wie ein neuer Lauf aus. NOLOAD und ein einzelner Initialisierer bleiben der Vertrag aus Vorlesung 6: das andere Startup löscht den Block nicht ein zweites Mal. HSEM ersetzt keine Cache-Kohärenz. Die nicht cachebare Region ersetzt keine MPU. Die Tabelle ist die Nachschlagehilfe. Die lokale Erklärung steht auf der Kriterien- und auf der Messfolie.